

Tutorial 7

1. Compute KMP-table array h for pattern "babbaabba".

Solution: The algorithm at the end is given. In the beginning, index i is 1, and index j is 0. Note that the length m is 9 and the array h looks like this:

-1									
----	--	--	--	--	--	--	--	--	--

Now, $pat[1]$ is 'a' while $pat[0]$ is 'b'. Since the two are not equal, we assign $h[1]$ to 0 (j).

-1	0								
----	---	--	--	--	--	--	--	--	--

The while loop sets $j = h[j] = -1$. Then, both get incremented so that $i = 2$ and $j = 0$. Now, $pat[2] = pat[0]$ so we assign $h[2] = h[0] = -1$.

-1	0	-1							
----	---	----	--	--	--	--	--	--	--

Now, after incrementing and in the next iteration, $pat[3] \neq pat[1]$ so $h[3]$ is assigned to 1.

-1	0	-1	1						
----	---	----	---	--	--	--	--	--	--

One iteration of the (inner) while loop assigns j to $h[j]$ which is 0. Then, $pat[3] = pat[0]$ so it stops there. After incrementing both variables, we see that $pat[4] = pat[1]$ so we assign $h[4]$ to $h[1]$ which is 0.

-1	0	-1	1	0					
----	---	----	---	---	--	--	--	--	--

On incrementing both variables, $pat[5] \neq pat[2]$ so we assign $h[5]$ to 2.

-1	0	-1	1	0	2				
----	---	----	---	---	---	--	--	--	--

One iteration of the while loop assigns j to $h[j]$ which is -1. After incrementing, we get $i = 6$ and $j = 0$. Now, $pat[6] = pat[0]$ so $h[6]$ is assigned to $h[0]$ which is -1.

-1	0	-1	1	0	2	-1			
----	---	----	---	---	---	----	--	--	--

Next iteration, $pat[7] \neq pat[1]$ so $h[7]$ is assigned to 1.

-1	0	-1	1	0	2	-1	1		
----	---	----	---	---	---	----	---	--	--

The while loop resets j to -1, following which j is incremented to 0 and i to 8. Now, $pat[8] \neq pat[0]$ so $h[8]$ gets assigned to zero.

-1	0	-1	1	0	2	-1	1	0	
----	---	----	---	---	---	----	---	---	--

At last, $j = 0$ and $i = 9$. Now, since $i = m$ we break the standard loop and assign $h[i]$ to j , which is zero. At the end, the KMP table looks like this:

-1	0	-1	1	0	2	-1	1	0	0
----	---	----	---	---	---	----	---	---	---

Algorithm 1: KMPtable

```
1 Function KMPtable(String pat):
2    $i \leftarrow 1$ ;
3    $j \leftarrow 0$ ;
4    $m \leftarrow \text{pat.length}()$ ;
5    $h \leftarrow \text{new int } [n + 1]$ ;
6    $h[0] \leftarrow -1$ ;
7   while  $i < m$  do
8     if  $\text{pat}[i] \neq \text{pat}[j]$  then
9        $h[i] \leftarrow j$ ;
10      while  $j \geq 0$  and  $\text{pat}[i] \neq \text{pat}[j]$  do
11         $j \leftarrow h[j]$ ;
12      end
13    end
14    else
15       $h[i] \leftarrow h[j]$ ;
16    end
17     $i \leftarrow i + 1$ ;
18     $j \leftarrow j + 1$ ;
19  end
20   $h[m] \leftarrow j$ ;
21  return  $h$ ;
22 end
```

2. Is this version of the KMPtable algorithm correct?

Algorithm 2: KMPtableDiff

```
1 Function KMPtableDiff(String pat):
2    $i \leftarrow 1$ ;
3    $j \leftarrow 0$ ;
4    $m \leftarrow \text{pat.length}()$ ;
5    $h \leftarrow \text{new int } [n + 1]$ ;
6    $h[0] \leftarrow -1$ ;
7   while  $i < m$  do
8      $h[i] \leftarrow j$ ;
9     while  $j \geq 0$  and  $\text{pat}[i] \neq \text{pat}[j]$  do
10       $j \leftarrow h[j]$ ;
11    end
12     $i \leftarrow i + 1$ ;
13     $j \leftarrow j + 1$ ;
14  end
15   $h[m] \leftarrow j$ ;
16  return  $h$ ;
17 end
```

Solution: The answer is **no**. This algorithm is incorrect. Whenever $\text{pat}[i] = \text{pat}[j]$, the wrong value is inserted into the KMP table (meaning the table is incorrect). This was discussed in slides that if $h[i]$ is assigned to j , it does not meet the requirement that when the shifted string pat is aligned with pat , the compared indices i and j differ. If they do not, then the main search algorithm gets more inefficient.

3. Suppose that there is a letter z in pat of length n such that it occurs in only one place, say k , which is given in advance. Can we optimize the computation of h ?

Solution: Let the character z appear in position $m < n$ and nowhere else. Recall the function of the table is to find the largest index j given i such that $pat[0:j] = pat[i-j:i]$ but $pat[j] \neq pat[i]$. By definition, that j has to be less than i , else it implies a shift of the search string by 0 in the main search (since in case of inequality between $txt[i]$ and $pat[j]$, there, we move j to $h[j]$ and large $h[j]$ means small shift forward; $j = h[j]$ means no shift).

If $j > m$, it means that $pat[0:j]$ contains z . For the equality to hold, $pat[i-j:i]$ should contain z at index m , meaning that $pat[j+m] = z$, which goes against the premise of our question: there is only one index where z is located. So, we say that j in the computation can never exceed m or, more specifically, no entry in the KMP table can exceed m . Further, $pat[i-j:i]$ cannot contain z or index m in its range.

The computation of the KMP table till index m is the same as the earlier algorithm, and $h[m]$ will get assigned to the correct value anyway. For higher values, we split pat into $subpat = pat[0:m]$ and $src = pat[m+1:n]$. In order to compute the value of $h[i]$ for $i > m$, we try to follow the KMP search algorithm with src as the text and $subpat$ as its pattern. Except, we do not look for exact matches. Rather, when $src[i] \neq subpat[j]$, we assign $h[i+m+1]$ to j , as expected.

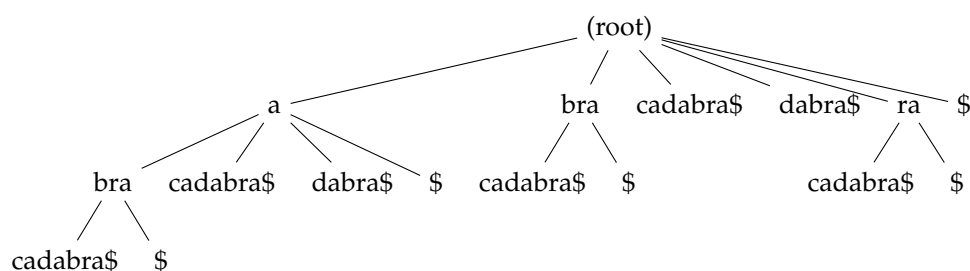
The case when $src[i] = subpat[j]$ seems a bit complicated, but just like the KMP table computation, doing $h[i+m+1] \leftarrow h[j]$ will work, since $src[i] = subpat[h[j]] \neq subpat[j]$ and all previous indices are equal. When $src[i] \neq subpat[j]$, after assigning $h[i+m+1] \leftarrow j$, we keep moving j much like KMPsearch and KMPtable algorithms.

This begs the question: is it really more efficient? To begin with, we cannot achieve less than $O(n)$ since we need to parse the entire string at least once (the output depends on all characters) and generate an $O(n)$ output. But can we make it faster $O(n)$? Such as reducing the constant c rather than focusing on asymptotic complexity?

One idea is to multi-thread the two computations, the KMP-table of $subpat$ and the modified KMP-search of $subpat$ in src that just updates the same table (algorithm not shown, since it gets cluttered and tedious with managing threads, and guaranteeing that i in the $subpat$ thread is not less than j in the KMPsearch – this requires locks and semaphores knowledge, and can still be as ‘inefficient’ which is $O(n)$). Any single-threaded solution (such as doing the 2 procedures in sequence or in one master function with somewhat on-demand computation of $h[j]$ for j in $subpat$) necessarily takes as much time as the normal KMPtable-filling algorithm.

4. Compute the suffix tree for ‘*abracadabra\$*’. Compress degree 1 nodes. Use substrings as edge labels. Put a square around nodes where a word ends. Use it to locate the occurrences of ‘*abr*’

Solution: This is the required suffix tree (dollar (\$) – where a word ends):



To locate 'abr', we traverse the suffix tree to node for 'a'. Then we look at the child corresponding to 'br'. It is the left child. We see that there are two suffixes: 'abra' (just that, ending in \$ symbol node) and 'abracadabra', which is the whole word itself.

5. Review the argument that for a given text T , consisting of k words, the ordinary trie occupies space, which is a constant multiple of $\|T\|$. How is it that the suffix tree for a text T is of size $O(\|T\|^2)$? Give a worst-case example.

Solution: Yes, for a given text, the trie occupies space a constant multiple of $n = \|T\|$, since each character has almost one node associated with it, and the space taken by each node in the trie is constant. Even if the alphabet is infinite, a different representation of trie (linked list for children) implies that each trie-node has at max one parent, and therefore, the space taken is directly proportional to the number of nodes.

On the other hand, the suffix tree stores **all** suffixes. Consider the text T having only one word: each suffix (corresponding to a character in the word, of the n) takes up $O(\text{size of suffix})$ space. The net space evaluates to $O(n^2)$ since suffixes are substrings of length $1, 2, \dots, n$ where n is the length of one word.

Sure, there may be patterns that are repeated (and therefore, the number of nodes is fewer), and the best case is $O(n)$ for strings like $aaaaaa \dots aa$. However, the worst case is $O(n^2)$ when all characters are distinct (particularly when there is no restriction on character space), and the text comprises one word.